GATE in Data Science and AI Study Materials
PDSA
By Piyush Wairale

Instructions:

- Kindly go through the lectures/videos on our website www.piyushwairale.com
- Read this study material carefully and make your own handwritten short notes. (Short notes must not be more than 5-6 pages)
- Attempt the question available on portal.
- Revise this material at least 5 times and once you have prepared your short notes, then revise your short notes twice a week
- **If you are not able to understand any topic or required detailed explanation, please mention it in our discussion forum on webiste**
- **Let me know, if there are any typos or mistake in study materials. Mail me at piyushwairale100@gmail.com**

For GATE DA Crash Course, visit: www.piyushwairale.com

1 Linear Search and Binary Search Algorithms in Python

1.1 Linear Search

- Linear search, also known as sequential search, is a simple search algorithm that looks for a target value within a list, one element at a time.
- Linear search is a method of finding elements within a list. It is also called a sequential search. It is the simplest searching algorithm because it searches the desired element in a sequential manner.
- It compares each and every element with the value that we are searching for. If both are matched, the element is found, and the algorithm returns the key's index position.

1.1.1 Algorithm:

1. Start from the beginning of the list.
2. Compare each element with the target value.
3. If the element is equal to the target, return its index.
4. If the end of the list is reached without finding the target, return a special value (e.g., -1) to indicate that the target is not in the list.

1.1.2 Python Implementation:

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

1.1.3 Example:

```
my_list = [2, 4, 7, 1, 9, 6]
target_value = 7
result = linear_search(my_list, target_value)
print(f"Index of {target_value}: {result}")
```


For GATE DA Crash Course, visit: www.piyushwairale.com

1.2 Binary Search

- A binary search is an algorithm to find a particular element in the list. Suppose we have a list of thousand elements, and we need to get an index position of a particular element. We can find the element's index position very fast using the binary search algorithm.
- There are many searching algorithms but the binary search is most popular among them.
- The elements in the list must be sorted to apply the binary search algorithm. If elements are not sorted then sort them first.
- Binary search is an efficient search algorithm that finds the position of a target value within a sorted array.
- In the binary search algorithm, we can find the element position using the following methods:
  1. **Recursive Method**
  2. **Iterative Method**
- The divide and conquer approach technique is followed by the recursive method. In this method, a function is called itself again and again until it found an element in the list.
- A set of statements is repeated multiple times to find an element's index position in the iterative method. The while loop is used for accomplish this task.
- Binary search is more effective than linear search because we don't need to search each list index. The list must be sorted to achieve the binary search algorithm.

1.2.1 Algorithm:

1. Start with the entire sorted array.
2. Set the lower bound to the first index and the upper bound to the last index.
3. Compute the middle index: `mid = (lower + upper) // 2`.
4. If the middle element is equal to the target, return its index.
5. If the target is less than the middle element, search in the left half. Otherwise, search in the right half.
6. Repeat the process until the target is found or the bounds overlap.

For GATE DA Crash Course, visit: www.piyushwairale.com

1.2.2 Python Implementation:

```
def binary_search(arr, target):
    lower, upper = 0, len(arr) - 1
    while lower <= upper:
        mid = (lower + upper) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lower = mid + 1
        else:
            upper = mid - 1
    return -1
```

1.2.3 Example:

```
sorted_list = [1, 2, 4, 6, 7, 9]
target_value = 7
result = binary_search(sorted_list, target_value)
print(f"Index of {target_value}: {result}")
```


For GATE DA Crash Course, visit: www.piyushwairale.com

1.3 Comparison: Linear Search vs. Binary Search

Basis of Comparison	**Linear Search**	**Binary Search**
Definition	The linear search starts searching from the first element and compares each element with a searched element till the element is not found.	It finds the position of the searched element by finding the middle element of the array.
Sorted data	In a linear search, the elements don't need to be arranged in sorted order.	The pre-condition for the binary search is that the elements must be arranged in a sorted order.
Implementation	The linear search can be implemented on any linear data structure such as an array, linked list, etc.	The implementation of binary search is limited as it can be implemented only on those data structures that have two-way traversal.
Approach	It is based on the sequential approach.	It is based on the divide and conquer approach.
Size	It is preferable for small-sized data sets.	It is preferable for large-size data sets.
Efficiency	It is less efficient in the case of large-size data sets.	It is more efficient in the case of large-size data sets.
Worst-case scenario	In a linear search, the worst-case scenario for finding the element is $O(n)$.	In a binary search, the worst-case scenario for finding the element is $O(\log_2 n)$.
Best-case scenario	In a linear search, the best-case scenario for finding the first element in the list is $O(1)$.	In a binary search, the best-case scenario for finding the first element in the list is $O(1)$.
Dimensional array	It can be implemented on both a single and multidimensional array.	It can be implemented only on a multidimensional array.

For GATE DA Crash Course, visit: www.piyushwairale.com

2 Basic Sorting Algorithms

A sorting algorithm is a method for arranging the elements of a list or collection in a specific order. Sorting is a fundamental operation in computer science and is applied to various tasks, such as searching, data analysis, and optimization. Sorting algorithms play a crucial role in organizing data efficiently, and there are various algorithms designed to achieve this task. These algorithms differ in their approaches, time complexity, and space complexity.

2.0.1 Purpose of sorting

- Sorting is a technique which reduces problem complexity and search complexity.
- Insertion sort takes $O(n^2)$ time in the worst case. It is a fast inplace sorting algorithm for small input sizes.
- Merge sort has a better asymptotic running time $O(nlogn)$, but it does not operate in inplace.
- Heap sort, sorts 'n' numbers inplace in $O(nlogn)$ time, it uses a data structure called heap, with which we can also implement a priority queue.
- Quick sort also sorts 'n' numbers in place, but its worst – case running time is $O(n^2)$. Its average case is $O(nlogn)$. The constant factor in quick sort's running time is small, This algorithm performs better for large input arrays.
- Insertion sort, merge sort, heap sort, and quick sort are all comparison-based sorts; they determine the sorted order of an input array by comparing elements

2.1 Selection Sort

- Selection sort is a sorting algorithm, specifically an in-place comparison sort. It has $O(n^2)$ complexity, making it inefficient on large lists.
- Selection Sort is another simple sorting algorithm. It divides the input list into a sorted and an unsorted region. The algorithm repeatedly selects the smallest (or largest, depending on the order) element from the unsorted region and swaps it with the first element of the unsorted region. The process is repeated until the entire list is sorted.
- **Time complexity:** $O(n^2)$ in all cases (worst, average, and best)
- **Space complexity:** $O(1)$
- **Basic idea:** Find the minimum element in the unsorted portion of the array and swap it with the first unsorted element. Repeat until the array is fully sorted.
- **Advantages:** Simple implementation, works well for small datasets, requires only constant space, in-place sorting algorithm

For GATE DA Crash Course, visit: www.piyushwairale.com

- **Disadvantages:** Inefficient for large datasets, worst-case time complexity of $O(n^2)$, not optimal for partially sorted datasets, not a stable sorting algorithm

2.1.1 Algorithm:

1. Find the minimum element in the unsorted part of the array.
2. Swap it with the first element of the unsorted part.
3. Move the boundary between the sorted and unsorted parts.
4. Repeat until the entire array is sorted.

2.1.2 Python Implementation:

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
```

2.2 Bubble Sort

- Bubble sort is a simple sorting algorithm that works by repeatedly stepping through the list to be sorted, comparing each pair of adjacent items, and swapping them if they are in the wrong order. The pass through the list is repeated until no swaps are needed, which indicates that the list is sorted. The algorithm gets its name from the way smaller elements 'bubble' to the top of the list.
- **Time complexity:** $O(n^2)$ in the worst and average cases, $O(n)$ in the best case (when the input array is already sorted)
- **Space complexity:** $O(1)$
- **Basic idea:** Iterate through the array repeatedly, comparing adjacent pairs of elements and swapping them if they are in the wrong order. Repeat until the array is fully sorted.
- **Advantages:** Simple implementation, works well for small datasets, requires only constant space, stable sorting algorithm
- **Disadvantages:** Inefficient for large datasets, worst-case time complexity of $O(n^2)$, not optimal for partially sorted datasets

For GATE DA Crash Course, visit: www.piyushwairale.com

2.2.1 Algorithm:

1. Compare each pair of adjacent elements in the array.
2. Swap them if they are in the wrong order.
3. Repeat until no more swaps are needed.

2.2.2 Python Implementation:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

2.3 Insertion Sort

- Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time. It is much less efficient on large lists than more advanced algorithms, such as quicksort, heapsort, or merge sort. However, it performs well for small datasets or partially sorted datasets.
- **Time complexity:** $O(n^2)$ in the worst and average cases, $O(n)$ in the best case (when the input array is already sorted)
- **Space complexity:** $O(1)$
- **Basic idea:** Build up a sorted subarray from left to right by inserting each new element into its correct position in the subarray. Repeat until the array is fully sorted.
- **Advantages:** Simple implementation, works well for small datasets, requires only constant space, efficient for partially sorted datasets, stable sorting algorithm
- **Disadvantages:** Inefficient for large datasets, worst-case time complexity of $O(n^2)$

2.3.1 Algorithm:

1. Build a sorted array one element at a time.
2. Take each element and insert it into its correct position in the sorted array.

For GATE DA Crash Course, visit: www.piyushwairale.com

2.3.2 Python Implementation:

```
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
```


For GATE DA Crash Course, visit: www.piyushwairale.com

3 Divide and Conquer Algorithms: Mergesort and Quicksort

- Divide-and-conquer is a top down technique for designing algorithms that consists of dividing the problem into smaller sub problems hoping that the solutions of the sub problems are easier to fi nd and then composing the partial solutions into the solution of the original problem.
- Divide-and-conquer paradigm consists of following major phases:
  - Breaking the problem into several sub-problems that are similar to the original problem but smaller in size.
  - Solve the sub-problem recursively (successively and independently)
  - Finally, combine these solutions to sub-problems to create a solution to the original problem
- Divide and conquer is a powerful algorithmic paradigm that involves breaking a problem into subproblems, solving them independently, and combining their solutions to solve the original problem.

  This technique can be divided into the following three parts:
  Divide: This involves dividing the problem into smaller sub-problems.
  Conquer: Solve sub-problems by calling recursively until solved.
  Combine: Combine the sub-problems to get the final solution of the whole problem.

Divide-and-Conquer Examples

- Sorting: Merge sort and quick sort
- Binary tree traversals
- Binary Search
- Multiplication of large integers
- Matrix multiplication: Strassen's algorithm
- Closest-pair and Convex-hull algorithm

3.1 Mergesort

Merge Sort is also a sorting algorithm. The algorithm divides the array into two halves, recursively sorts them, and finally merges the two sorted halves.

For GATE DA Crash Course, visit: www.piyushwairale.com

3.1.1 Algorithm:

1. **Divide:** Divide the unsorted list into n sublists, each containing one element.
2. **Conquer:** Repeatedly merge sublists to produce new sorted sublists until there is only one sublist remaining.
3. **Combine:** The final result is a sorted list.

3.1.2 Python Implementation:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        merge_sort(left_half)
        merge_sort(right_half)

        i = j = k = 0

        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1
```


For GATE DA Crash Course, visit: www.piyushwairale.com

3.2 Quicksort

Quicksort is a sorting algorithm. The algorithm picks a pivot element and rearranges the array elements so that all elements smaller than the picked pivot element move to the left side of the pivot, and all greater elements move to the right side. Finally, the algorithm recursively sorts the subarrays on the left and right of the pivot element.

3.2.1 Algorithm:

1. **Divide:** Choose a pivot element and partition the array into two subarrays such that elements smaller than the pivot are on the left, and elements greater than the pivot are on the right.
2. **Conquer:** Recursively apply quicksort to the subarrays.
3. **Combine:** The final result is a sorted array.

3.2.2 Python Implementation:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)
```

3.3 Comparison:

- **Mergesort:**
  - Stable.
  - Time Complexity: $O(n \log n)$ in all cases.
  - Space Complexity: $O(n)$ additional space.
- **Quicksort:**
  - Unstable.
  - Time Complexity: $O(n \log n)$ average case, $O(n^2)$ worst case (with poor pivot selection).
  - Space Complexity: $O(\log n)$ additional space.

For GATE DA Crash Course, visit: www.piyushwairale.com

References

- Pearson's GATE CS Notes
- geeksforgeeks.org/
- javatpoint.com/
- w3schools.com/
- tutorialspoint.com/
- byjus.com

For GATE DA Crash Course, visit: www.piyushwairale.com